ABC Aprendiz



A ABC Aprendiz atua na região do ABC Paulista oferecendo formação pessoal e profissional para jovens e adolescentes, além de apoiar empresas na contratação de aprendizes. No entanto, administrar a entrada de centenas de novos alunos nos programas ofertados é um desafio logístico. Todo semestre, a equipe precisa realizar o 'ensalamento' (distribuição dos alunos em turmas) de forma totalmente manual, o que demanda muito tempo e gera retrabalho. Agora, é a nossa vez de dar o próximo passo: **criar um sistema automatizado capaz de resolver esse quebra-cabeça e entregar turmas perfeitamente balanceadas e organizadas.**

Desafio UniSoma 2026: Automação para Otimizar a Educação

A tecnologia tem o poder de revolucionar a gestão escolar. Neste desafio, você vai desenvolver um programa inteligente capaz de alocar centenas de alunos matriculados em turmas da Manhã e da Tarde, respeitando regras de capacidade e equidade. A ideia é um sistema que transforma horas de trabalho em um processo de poucos cliques, garantindo turmas equilibradas e permitindo que a equipe da ONG foque no que realmente importa: a educação dos jovens.

Especificações do Projeto

- Trabalho em **dupla, trio ou quarteto**, simulando dinâmicas reais de trabalho em equipe.
- Grupos podem trabalhar em formato presencial, mas o hackathon será **remoto**.
- Linguagem de programação: **Python==3.12.9**
- **Abordagem de Resolução: É estritamente necessário a utilização de uma modelagem matemática para a resolução do problema. Soluções baseadas em heurísticas/loops não serão aceitas**.

Entregáveis

1. **Executável (.exe)** que roda diretamente no computador da ONG, sem necessidade de instalação de ambientes de programação.
2. **Manual de Instruções** em formato .pdf explicando ao usuário como utilizar a solução.
3. **Arquivos de Saída:** Planilha de ensalamento (.xlsx) exatamente na ordem e com as colunas do template
    a. **A entrega de um Relatório Resumo (.pdf/.txt/dash)** será considerada um diferencial na avaliação.
4. **Código py** de ETL e modelagem para verificarmos se o modelo utilizou algoritmos de otimização (não pode ser heurística)
5. **Apresentação:** até 3 slides para "vender" seu app para a banca na apresentação final do hackathon.

Regras de Negócio

- **Disponibilidade**: Alunos de manhã deverão apenas ir para turmas da manhã; a mesma lógica se aplica para os alunos da tarde
- **Equilíbrio em Quantidade:** As turmas de um mesmo turno não podem ser desproporcionais (ex: evitar uma turma com 5 e outra com 20 alunos).
- **Equilíbrio de Gênero:** Cada turma de um mesmo turno deve manter uma proporção similar entre meninos (M) e meninas (F).
- **Capacidade Máxima:** O limite de vagas deve ser respeitado
- **Nomenclatura das Turmas:** O sistema deve nomear as turmas em sequência alfabética contínua. As da manhã recebem as primeiras letras (ex.: MANHÃ - A, MANHÃ - B) e as da tarde continuam a sequência (ex.: TARDE - C, TARDE - D).
- Todos os alunos devem ser ensalados: alertar quando não é possível ensalar todos, quando o número de alunos no turno é maior que o número de vagas totais no turno.
- Podemos ter alunos com exatamente o mesmo nome na lista de matriculados gerando "duplicidade": alertar pois pode ser erro de cadastro, no entanto sabemos que é possível ter pessoas distintas com exatamente o mesmo nome, alocando então em "duplicidade".

Input

A instituição preencherá um excel com a relação dos matriculados e as capacidades das turmas:

1. **Aba Alunos:** contendo os dados de Nome do Aluno, Sexo (M/F) e Disponibilidade (Manhã/Tarde). O input pode conter erros de preenchimento (por exemplo, "manha" ao invés de "Manhã") que devem ser tratados pelo programa.

Nome	Sexo	Disponibilidade
Gustavo de Oliveira Cardoso	M	Manhã
Daniel Souza Gomes	M	
Camila Machado do Nascimento	F	Tarde
Theo Teixeira Carvalho Martins	M	manha
Lara Dias Ferreira Rodrigues	F	Manhã
Caio de Andrade da Costa	x	Tarde
Rafaela Rodrigues Rocha	F	Manhã
Murilo Ferreira Cardoso Cavalcanti	M	Tarde
Enzo de Oliveira Rodrigues Machado		Tarde
Isabela Machado Souza	F	Manhã
Arthur Souza do Nascimento Dias	M	Manhã
João de Jesus Batista Lopes	M	Tarde

2. **Aba Turmas:** uma tabela simples informando a estrutura disponível, e todas as turmas do mesmo turno terão o mesmo número de vagas.

Recursos	Quantidade
turmas_manha	2
turmas_tarde	2
vagas_manha	35
vagas_tarde	35

Output

A interface da aplicação fica para a criatividade do grupo, sendo obrigatório a seleção do arquivo excel com o cenário pela interface. A aplicação deverá gerar o ensalamento dos alunos nas turmas, salvando o resultado em duas novas abas do mesmo excel de input:

➔ Aba **Ensalamento**: tabela contendo as colunas Nome do aluno, Sexo, Disponibilidade e Turma (com o nome formatado no padrão [PERÍODO] - [LETRA]). Veja o exemplo de output abaixo

Nome	Sexo	Disponibilidade	Turma
Júlia Ribeiro Pereira Monteiro	TARDE	F	TARDE - D
Murilo Monteiro Moreira	MANHÃ	M	MANHÃ - C
Giovanna do Nascimento Soares Nunes	TARDE	F	TARDE - F
Heitor Mendes Lima	TARDE	M	TARDE - F
Arthur Martins Rodrigues Monteiro	MANHÃ	M	MANHÃ - B
Vitória Rodrigues Nunes Monteiro	TARDE	F	TARDE - F
Gabriel do Nascimento dos Santos de Oliveira	MANHÃ	M	MANHÃ - B

➔ Aba **Inconsistências**: motivo de inconsistências que foram tratadas ou que precisam ser tratadas para gerar o ensalamento. O ideal é alertar o usuário de todas as inconsistências que impossibilitam a execução do ensalamento de uma só vez, senão criará um loop de tentativas de execução em cenários com muitos dados incorretos.

Temos quatro cenários para teste do programa e geração dos outputs, mas os grupos podem criar outros cenários, buscando criar situações reais que podem “quebrar” a aplicação ou ensalamento.

Um programa **Validador.exe**, de validação de outputs, disponibilizado irá ajudar a testar os resultados das abas outputs, que devem ter sido geradas com o executável do grupo. O programa verificará se:

- ➔ o formato de entrega foi violado
- ➔ as restrições de negócio foram violadas (capacidade, horário, nome das turmas etc.)
- ➔ inconsistências estão sendo checadas pelo programa principal

● **Relatório Resumo da Distribuição (.pdf/.txt/dash) [Diferencial]:** podemos ter um diagnóstico mostrando o total de alunos alocados, total por turma e a proporção de gênero alcançada.

Dica para gerar o executável usando pyinstaller

O pyinstaller é uma biblioteca do Python para empacotar e criar um exe, mas não é a única biblioteca possível. Não é obrigatório utilizá-la, o importante é criar um executável para a entrega, facilitando o uso da aplicação pela ONG.

1. Instale as dependências: certifique-se de ter o pyinstaller instalado
2. Crie um script principal app.py
3. Use o PyInstaller para empacotar

    Execute o seguinte comando: **pyinstaller --onefile run_app.py**
    --onefile: gera um único executável .exe
    O executável será criado na pasta dist/
    Para incluir arquivos adicionais (como imagens, CSVs, etc.), use a flag --add-data

4. Execute o aplicativo: depois de gerado, vá até a pasta dist e execute ./run_app.exe Isso abrirá o navegador com seu app rodando localmente.

Avaliação Técnica

Executaremos a aplicação entregue para mais 11 instâncias e avaliaremos o resultado para classificar os 10 grupos que farão a apresentação final segundo os critérios abaixo:

- **Entrega:** O executável foi entregue, funciona sem necessidade de linha de comando e gera os relatórios esperados?
- **Modelagem**: Utiliza bibliotecas de modelagem matemática para gerar a solução?
- **Eficácia do Balanceamento:** A solução dividiu bem os alunos e manteve a proporção de gênero equilibrada nas turmas?
- **Respeito às restrições:** O programa cumpriu as regras de horário, limite de vagas e nomenclatura corretamente?
- **Usabilidade:** A interface é intuitiva, prática e acessível para a equipe da ABC Aprendiz executar no dia a dia?
- **Checagem de inconsistências**: o programa alerta inconsistências corretamente? Como trata?

Diferencial: Conforme mencionado, a geração e entrega do Relatório Resumo da Distribuição será avaliada como um diferencial para o projeto.